

Analisis teorico del proyecto

Planificador de rutas en una red de transporte

1. Descripcion general

El proyecto implementa un programa en Python que modela una red de estaciones como un grafo ponderado. Cada estacion es un nodo y cada conexion entre estaciones es una arista con un peso asociado, que representa el tiempo en minutos necesario para ir de una estacion a otra.

El programa permite cargar una red desde archivo, anadir estaciones, crear conexiones, mostrar la red, comprobar si dos estaciones estan conectadas, calcular la ruta mas rapida con Dijkstra y guardar los datos para futuras ejecuciones.

2. Representacion del grafo

La red se representa mediante listas de adyacencia usando diccionarios. Cada clave principal es una estacion y su valor es otro diccionario con sus estaciones vecinas y el tiempo de conexion.

```
{
  "Atocha": {"Sol": 5, "Chamartin": 12},
  "Sol": {"Atocha": 5, "Gran Via": 3}
}
```

Esta representacion es adecuada porque solo guarda las conexiones existentes, por lo que es mas eficiente que una matriz de adyacencia cuando no todas las estaciones estan conectadas entre si.

3. Estructuras de datos utilizadas

Diccionarios: almacenan el grafo y permiten buscar, insertar o modificar estaciones y conexiones con coste medio $O(1)$.

Conjuntos: guardan las estaciones visitadas en BFS/DFS. Sirven para no repetir nodos y evitar ciclos infinitos. Comprobar pertenencia tiene coste medio $O(1)$.

Cola: se usa en BFS para explorar las estaciones por niveles, siguiendo el criterio FIFO: el primer elemento que entra es el primero que sale.

Heap: se usa en Dijkstra mediante heapq para obtener de forma eficiente la estacion con menor distancia acumulada.

Archivos: permiten cargar y guardar la red, dando persistencia a los datos para no perder la informacion al cerrar el programa.

4. Algoritmos principales

BFS o busqueda en anchura se utiliza para comprobar si existe camino entre dos estaciones. Empieza en una estacion origen, visita primero sus vecinos directos y despues avanza por niveles. No tiene en cuenta los pesos, por lo que sirve para conectividad, no para ruta mas rapida.

DFS o busqueda en profundidad explora un camino hasta el final antes de retroceder. Tambien sirve para comprobar si existe conexion entre estaciones, pero tampoco considera los pesos de las aristas.

Dijkstra calcula la ruta mas rapida entre dos estaciones en un grafo ponderado con pesos no negativos. En este proyecto los pesos son minutos, por lo que no existen valores negativos. El algoritmo mantiene distancias acumuladas, predecesores para reconstruir el camino y una cola de prioridad para seleccionar siempre la estacion mas cercana pendiente de procesar.

5. Ruta con estacion intermedia

Una mejora incluida consiste en calcular la ruta mas rapida pasando obligatoriamente por una estacion intermedia. Para hacerlo se aplica Dijkstra dos veces: primero desde el origen hasta la estacion intermedia y despues desde la estacion intermedia hasta el destino. Finalmente se unen ambas rutas y se suma el tiempo total.

Aunque se ejecuta Dijkstra dos veces, en notacion Big O la complejidad se mantiene como $O((V + E) \log V)$, ya que las constantes se eliminan. En la practica tarda aproximadamente el doble que una unica ejecucion de Dijkstra.

6. Control de errores y validaciones

El programa debe controlar entradas incorrectas para evitar fallos durante la ejecucion. Entre las validaciones importantes estan: no permitir estaciones duplicadas, comprobar que las estaciones existen antes de crear una conexion, evitar tiempos negativos o no numericos, controlar archivos inexistentes o mal formateados y evitar conexiones duplicadas.

El uso de try/except permite capturar errores y mostrar mensajes comprensibles al usuario sin que el programa se cierre inesperadamente.

7. Complejidad temporal

Operacion	Complejidad	Explicacion
Anadir estacion	$O(1)$	Busqueda e insercion en diccionario en coste medio constante.
Anadir conexion	$O(1)$	Acceso directo a origen y destino en el diccionario.
Mostrar estaciones	$O(V)$	Se recorren todas las estaciones.
Mostrar conexiones	$O(V + E)$	Se recorren estaciones y conexiones existentes.
Comprobar conectividad	$O(V + E)$	BFS/DFS puede recorrer todo el grafo.
Ruta mas rapida	$O((V + E) \log V)$	Dijkstra con heap o cola de prioridad.
Cargar archivo	$O(E)$	Se leen las conexiones del archivo.
Guardar archivo	$O(V + E)$	Se guardan estaciones y conexiones.

8. Complejidad espacial

La complejidad espacial del grafo es $O(V + E)$, porque se almacenan todas las estaciones y todas las conexiones reales. Ademas, BFS, DFS y Dijkstra utilizan estructuras auxiliares como visitados, distancias, predecesores y colas, que pueden ocupar hasta $O(V)$. Por tanto, la complejidad espacial general se mantiene como $O(V + E)$.

9. Justificacion final

Se usa un grafo porque el problema representa una red de transporte. Las estaciones son nodos, las conexiones son aristas y los tiempos son pesos. La lista de adyacencia con diccionarios es una buena eleccion porque permite almacenar el grafo de forma compacta y acceder rapidamente a cada estacion.

BFS y DFS son adecuados para comprobar conectividad, mientras que Dijkstra es el algoritmo correcto para obtener la ruta mas rapida en un grafo ponderado con pesos positivos. La lectura y escritura de archivos permiten persistencia, y las validaciones hacen que el programa sea mas robusto.

10. Posibles mejoras

Como mejoras futuras se podrian anadir opciones para eliminar estaciones, eliminar conexiones, modificar tiempos, detectar estaciones aisladas, exportar un informe de la red, dibujar el grafo visualmente o usar un formato JSON para guardar los datos de manera mas estructurada.

Conclusion

El proyecto aplica conceptos esenciales de estructuras de datos y algoritmos: grafos ponderados, diccionarios, conjuntos, colas, heaps, BFS, DFS y Dijkstra. La solucion permite gestionar una red de estaciones y calcular rutas de forma eficiente, manteniendo los datos guardados entre ejecuciones y controlando errores habituales de entrada.